

ASSIGNMENT-4.4

K. Keerthan

2303A52008

Batch: 43

1. Sentiment Classification for Customer Reviews

Scenario:

An e-commerce platform wants to analyze customer reviews and classify them into Positive, Negative, or Neutral sentiments using prompt engineering.

Tasks:

Prepare 6 short customer reviews mapped to sentiment labels.

```
lab3.py > ...
1  # An e-commerce platform wants to analyze customer reviews and classify hem into Positive, Negative, or Neutral sentiments using prompt
2  # Prepare 6 short customer reviews mapped to sentiment labels.
3  from doctest import Example
4
5
6  reviews = [
7      ("The product quality is excellent and exceeded my expectations.", "Positive"),
8      ("I am very disappointed with the service I received.", "Negative"),
9      ("The item arrived on time and as described.", "Neutral"),
10     ("Fantastic experience! Will definitely shop here again.", "Positive"),
11     ("The packaging was damaged, but the product works fine.", "Neutral"),
12     ("Terrible quality, I want a refund immediately.", "Negative")
13 ]
```

*Design a Zero-shot prompt to classify sentiment.

```
16  # classify the statement of the review into Positive, Negative, or Neutral sentiment.
17  review = "The product is terrible and I'am very unhappy with my purchase."
18  sentiment = "Negative"
19
20  # classify the statement of the review into Positive, Negative, or Neutral sentiment.
21  review = "The product quality is excellent and exceeded my expectations."
22  sentiment = "Positive"
23  # classify the statement of the review into Positive, Negative, or Neutral sentiment.
24  review = "The item arrived on time and as described."
25  sentiment = "Neutral"
```

*Design a One-shot prompt with one labeled example.

```
26
27  # Classify the sentiment of the following customer reviews as 'Positive', 'Negative', or 'Neutral'.
28
29  # Example:
30  # Review: "This product is absolutely amazing! I highly recommend it."
31  # Sentiment: Positive
32  review = "Fantastic experience! Will definitely shop here again."
33  sentiment = "Positive"
34
35
```

*Design a Few-shot prompt with 3–5 labeled examples.

```

36
37 # Classify the sentiment of customer reviews as Positive, Negative, or Neutral.
38
39 # Examples:
40 # Review: "The product quality is amazing."
41 # Sentiment: Positive
42
43 # Review: "Very disappointed, waste of money."
44 # Sentiment: Negative
45
46 # Review: "It works fine, but nothing impressive."
47 # Sentiment: Neutral
48
49 # Review: "Fast delivery and great packaging."
50 # Sentiment: Positive
51
52 Review: "The packaging was damaged, but the product works fine."
53 Sentiment: Neutral

```

*Compare the outputs and discuss accuracy differences:

Few-shot prompting gives the highest accuracy because multiple labeled examples help the model learn patterns clearly. One-shot prompting improves accuracy compared to zero-shot by giving guidance through a single example. Zero-shot prompting is useful but less reliable, especially for neutral or mixed-sentiment reviews.

2. Email Priority Classification

Scenario:

A company wants to automatically prioritize incoming emails into High Priority, Medium Priority, or Low Priority.

Tasks:

*Create 6 sample email messages with priority labels.

```

50
57 # A company wants to automatically prioritize incoming emails into High Priority, Medium Priority, or Low Priority
58 # Create 6 sample email messages with priority labels.
59 emails = [
60     ("Please address this issue immediately, as it is affecting our operations.", "High Priority"),
61     ("Just a reminder about the upcoming team meeting next week.", "Low Priority"),
62     ("The quarterly report is due soon; please ensure all data is accurate.", "Medium Priority"),
63     ("Urgent: Server downtime reported, needs immediate attention!", "High Priority"),
64     ("FYI: New company policies have been updated on the intranet.", "Low Priority"),
65     ("Please review the attached document and provide feedback by Friday.", "Medium Priority")
66 ]

```

*Perform intent classification using Zero-shot prompting.

```

67
68 # classify the email into High Priority, Medium Priority, or Low Priority.
69 email = "This is to inform you about the scheduled maintenance this weekend."
70 priority = "Low Priority"

```

*Perform classification using One-shot prompting.

```

71
72 # classify the email into High Priority, Medium Priority, or Low Priority.
73 # email = "The client meeting has been rescheduled to tomorrow morning."
74 # priority = "Medium Priority"
75 email = "Our system is down and needs urgent fixing."
76 priority = "High Priority"

```

*Perform classification using Few-shot prompting.

```

77
78 # Classify the priority of the following email as 'High Priority', 'Medium Priority', or 'Low Priority'.
79 # Example:
80 # Email: "Please respond to this urgent request from the client."
81 # Priority: High Priority
82 # email = "The quarterly report is due soon; please ensure all data is accurate."
83 # priority = "Medium Priority"
84 # email = "Just a reminder about the upcoming team meeting next week."
85 # priority = "Low Priority"
86 email = "Urgent: Server downtime reported, needs immediate attention!"
87 priority = "High Priority"

```

*Evaluate which technique produces the most reliable results and why. ?

Few-shot prompting is likely to yield the most reliable results due to the presence of multiple 3abelled examples that help the model understand the context and nuances of email prioritization. One-shot prompting can also improve accuracy by providing a single example, but it may not capture the full range of scenarios as effectively as few-shot prompting. Zero-shot prompting, while useful, may struggle with accurately classifying emails without any prior examples, especially when distinguishing between Medium and Low Priority emails.

3. Student Query Routing System

Scenario:

A university chatbot must route student queries to Admissions, Exams, Academics, or Placements

Tasks:

*Create 6 sample student queries mapped to departments.

```

92
93 # A university chatbot must route student queries to Admissions, Exams, Academics, or Placements
94 # Create 6 sample student queries mapped to departments.
95 queries = []
96     ("How can I apply for the undergraduate program?", "Admissions"),
97     ("When will the exam results be announced?", "Exams"),
98     ("What courses are offered in the Computer Science department?", "Academics"),
99     ("Can you help me with internship opportunities?", "Placements"),
100     ("What are the admission requirements for international students?", "Admissions"),
101     ("Where can I find the academic calendar for this semester?", "Academics")
102 ]
103

```

*Implement Zero-shot intent classification using an LLM.

```

104 # classify the student query into Admissions, Exams, Academics, or Placements.
105 query = "I need information about scholarship opportunities."
106 department = "Admissions"
107 query = "When is the last date to submit exam applications?"
108 department = "Exams"
109 query = "What are the core subjects for the Mechanical Engineering course?"
110 department = "Academics"
111 query = "Are there any job fairs scheduled for this semester?"
112 department = "Placements"
113
114

```

*Improve results using One-shot prompting.

```

114 # Classify the following student query into 'Admissions', 'Exams', 'Academics', or 'Placements'.
115 # Example:
116 # Query: "How do I register for the final exams?"
117 # Department: Exams
118 query = "Can you help me with internship opportunities?"
119 department = "Placements"
120
121

```

*Further refine results using Few-shot prompting.

```

120
121 # classify the student query into Admissions, Exams, Academics, or Placements.
122 # Example:
123 # Query: "What are the admission requirements for international students?"
124 # Department: Admissions
125 # query = "Where can I find the academic calendar for this semester?"
126 # department = "Academics"
127 # query = "How can I apply for the undergraduate program?"
128 # department = "Admissions"
129 query = "When will the exam results be announced?"
130 department = "Exams"
131

```

*Analyze how contextual examples affect classification accuracy.:

Few-shot prompting is expected to provide the highest classification accuracy due to the presence of multiple labeled examples that help the model understand the context and nuances of student queries. One-shot prompting can also enhance accuracy by providing a single example, but it may not cover the full spectrum of possible queries as effectively as few-shot prompting. Zero-shot prompting, while useful, may face challenges in accurately classifying queries without any prior examples, especially when distinguishing between similar departments like Academics and Admissions.

4. Chatbot Question Type Detection

Scenario:

A chatbot must identify whether a user query is Informational, Transactional, Complaint, or Feedback

Tasks:

*Prepare 6 chatbot queries mapped to question types.

```

133
134 # A chatbot must identify whether a user query is Informational, Transactional, Complaint, or Feedback
135 # Prepare 6 chatbot queries mapped to question types
136 queries = []
137     ("What are your store hours?", "Informational"),
138     ("I want to return a defective product.", "Transactional"),
139     ("The delivery was late and the package was damaged.", "Complaint"),
140     ("I love the new features in your app!", "Feedback"),
141     ("How can I track my order?", "Informational"),
142     ("I'd like to change my shipping address.", "Transactional")
143

```

*Design prompts for Zero-shot, One-shot, and Few-shot learning

```

144
145 # Design prompts for Zero-shot, One-shot, and Few-shot learning
146 # classify the user query into Informational, Transactional, Complaint, or Feedback.
147 query = "The product I received is not working as expected."
148 question_type = "Complaint"
149 query = "How do I update my payment information?"
150 question_type = "Transactional"
151 query = "Your website is very user-friendly."
152 question_type = "Feedback"
153

```

```

154
155 # Classify the following user query into 'Informational', 'Transactional', 'Complaint', or 'Feedback'.
156 # Example:
157 # Query: "I want to cancel my order."
158 # Question Type: Transactional
159 query = "I love the new features in your app!"
160 question_type = "Feedback"

```

```

161 # Classify the user query into Informational, Transactional, Complaint, or Feedback.
162 # Example:
163 # query = "What are your store hours?"
164 # question_type = "Informational"
165 # query = "I'd like to change my shipping address."
166 # question_type = "Transactional"
167 # query = "The delivery was late and the package was damaged."
168 # question_type = "Complaint"
169 query = "How can I track my order?"
170 question_type = "Informational"
171 query = "I want to return a defective product."
172 question_type = "Transactional"

```

*Test all prompts on the same unseen queries:

Few-shot prompting is expected to yield the most reliable results due to multiple labeled examples that help the model understand different query types. One-shot prompting can improve accuracy by providing a single example, but may not cover all scenarios as effectively. Zero-shot prompting may struggle with accurately classifying queries without prior examples, especially when distinguishing between similar types like Informational and Feedback.

* Compare response correctness and ambiguity handling :

Few-shot prompting is likely to produce the most accurate and reliable classifications due to the presence of multiple examples that illustrate the distinctions between query types. This helps the model learn patterns and nuances more effectively. One-shot prompting can also enhance accuracy by providing a single example, but it may not capture the full range of scenarios as comprehensively as few-shot prompting. Zero-shot prompting, while useful for

quick classifications, may struggle with ambiguity and subtle differences between query types, leading to less reliable results. Overall, few-shot prompting is expected to outperform the other techniques in terms of correctness and handling ambiguous queries.

* Document observations :

Few-shot prompting is expected to provide the most accurate classifications due to the presence of multiple examples that illustrate the different query types. One-shot prompting can also enhance accuracy by providing a single example, but it may not cover the full spectrum of query types as effectively as few-shot prompting. Zero-shot prompting, while useful for quick classifications, may struggle with accurately identifying the nuances between similar query types without any prior examples.

5. Emotion Detection in Text

Scenario:

A mental-health chatbot needs to detect emotions: Happy, Sad, Angry, Anxious, Neutral

Tasks:

*Create labeled emotion samples.

```
177 |  
178 | # A mental-health chatbot needs to detect emotions: Happy, Sad, Angry, Anxious, Neutral  
179 |  
180 | emotions = [  
181 |     ("I am so excited about my new job!", "Happy"),  
182 |     ("I feel really down today.", "Sad"),  
183 |     ("Why did you do that? I'm furious!", "Angry"),  
184 |     ("I'm worried about the upcoming exam.", "Anxious"),  
185 |     ("It's just an average day, nothing special.", "Neutral"),  
186 |     ("I can't stop smiling after hearing the good news!", "Happy")  
187 | ]  
188 |
```

*Use Zero-shot prompting to identify emotions.

```
188 |  
189 | # classify the user statement into Happy, Sad, Angry, Anxious, or Neutral.  
190 | statement = "I am extremely frustrated with the delays."  
191 | emotion = "Angry"  
192 | statement = "I feel a bit nervous about the presentation."  
193 | emotion = "Anxious"  
194 | statement = "Today has been a pretty normal day."  
195 | emotion = "Neutral"  
196 |  
197 |
```

*Use One-shot prompting with an example

```

196
197 # Classify the following statement into 'Happy', 'Sad', 'Angry', 'Anxious', or 'Neutral'.
198 # Example:
199 # Statement: "I just got a promotion at work!"
200 # Emotion: Happy
201 statement = "I feel really down today."
202 emotion = "Sad"
203

```

***Use Few-shot prompting with multiple emotions.**

```

203
204 # classify the user statement into Happy, Sad, Angry, Anxious, or Neutral.
205 # Example:
206 # Statement: "I can't wait to see my friends this weekend!"
207 # Emotion: Happy
208 # statement = "It's just an average day, nothing special."
209 # emotion = "Neutral"
210 # statement = "Why did you do that? I'm furious!"
211 # emotion = "Angry"
212 statement = "I'm worried about the upcoming exam."
213 emotion = "Anxious"
214 statement = "I am so excited about my new job!"
215 emotion = "Happy"
216

```

***Discuss ambiguity handling across techniques :**

Few-shot prompting is expected to handle ambiguity better due to the presence of multiple examples that illustrate different emotional contexts. This helps the model learn subtle differences between similar emotions, such as Sad and Anxious. One-shot prompting provides some guidance but may not cover all nuances, leading to potential misclassifications in ambiguous cases. Zero-shot prompting relies solely on the model's pre-existing knowledge, which may not be sufficient for accurately distinguishing between closely related emotions, especially in complex or nuanced statements.